

BEC305 Python Programming — Assignment 5 —

Networked data, web services and databases

Module 5: Networked Programs, Web Services and Databases

Field	Value
Maximum marks	20
Weightage	10% of CIE
Issue date	<<TBD>>
Due date	<<TBD>>
Items	10 problems + 3 theory questions
CO / Bloom coverage	CO5: 13 · L1–L2 0, L3–L4 5, L5–L6 8

Objective

Build one working pipeline from network to query, and develop the habits that make it safe: checking status codes, keeping binary data binary, parameterising every SQL value, normalising before storing, and making a long job resumable.

Learning outcomes

- Student will be able to retrieve a resource over HTTP with both a raw socket and urllib, and explain what the library hides.
- Student will be able to parse XML and JSON into Python structures and handle absent fields safely.
- Student will be able to design a normalised schema with primary, logical and foreign keys.
- Student will be able to write parameterised SQL, including a JOIN across two tables.

Part A — Problems (17 marks)

P1. Fetch a text document using a raw socket. Print the status line, count the recv calls needed, and show what a single recv would have captured instead. [C05 | L4 | 2 marks]

P2. Download an image over HTTP, splitting headers from body at the first blank line. Verify the saved file size against Content-Length, then repeat with text mode and record exactly what went wrong. [C05 | L4 | 2 marks]

P3. Rewrite P1 using urllib and count the words in the document. State in a comment every protocol step that urlopen performed on your behalf, and the one thing it did not. [C05 | L3 | 2 marks]

P4. Write an XML document of at least five repeating records with one optional element. Parse it with findall, and show both the unguarded find().text crash and the guarded version. [C05 | L4 | 2 marks]

P5. Convert the same data to JSON, parse it, and produce identical output to P4. Compare the two parsers on lines of code and on handling of the optional field. [C05 | L4 | 2 marks]

P6. Design a two-table schema with a primary key, a UNIQUE logical key and a foreign key. Create it in SQLite, insert at least ten rows, and verify the schema in DB Browser with a screenshot. [C05 | L5 | 2 marks]

marks]

P7. Demonstrate a denormalised single table and its normalised two-table replacement holding the same data. Correct a repeated value in both and report how many cells each required. [C05 | L5 | 2 marks]

P8. Write a JOIN across your two tables. Then remove the ON clause, report the row count of each, and explain the multiplication. [C05 | L5 | 1 marks]

P9. Show a SQL injection against a string-formatted query using a value containing a quote, then show the same value handled safely by a ? placeholder. [C05 | L5 | 1 marks]

P10. Build the full pipeline — fetch, parse, store, query — against any open API. Make it resumable with a retrieved flag and idempotent with INSERT OR IGNORE, then prove the row count is unchanged after a second run. [C05 | L5 | 1 marks]

Part B — Theory (3 marks)

T1. Explain why HTTP was designed to be stateless, and what mechanisms applications use to reintroduce the state the protocol deliberately omits. [C05 | L5 | 1 marks]

T2. Compare XML and JSON for a data feed of your choice, naming one thing each does that the other cannot, and state which you would specify. [C05 | L5 | 1 marks]

T3. Argue why normalisation is worth a join on every read, and describe one situation in which you would deliberately denormalise. [C05 | L5 | 1 marks]

Submission guidelines

- Format: PDF, single file
- File name: <USN>_BEC305_Assignment5.pdf
- Channel: LMS upload
- Late policy: 10% per day, nothing after 5 days
- Every numerical answer must carry units and a stated reference direction.

Assessment rubric

Criterion	Weight	Excellent	Good	Satisfactory	Needs improvement
Network retrieval and byte handling	5	Status codes checked before any body is used, receive loop correct, binary data kept as bytes end to end, every encode and decode justified.	Retrieval correct with one gap in the byte handling.	Works for text but binary handling is shaky.	Single recv assumed sufficient, or binary body decoded as text.
Parsing XML and JSON	5	Both parsers used correctly, absent fields handled with None tests or get() defaults, findall preferred for collections with a stated reason.	Both parsers correct with minor gaps in absence handling.	Parsers work on the sample document only.	Chained .text on find(), or square-bracket access to optional JSON keys.

Criterion	Weight	Excellent	Good	Satisfactory	Needs improvement
Schema design and normalisation	6	Three key types used correctly, UNIQUE constraints present, denormalised and normalised versions compared quantitatively, schema verified in the browser.	Sound schema with a brief normalisation argument.	Tables created but keys used loosely.	Logical key used as the foreign key, or repeated strings left across rows.
SQL safety and the end-to-end pipeline	4	Every value parameterised, injection demonstrated and defended, JOIN correct with row counts checked, pipeline resumable and proven idempotent.	Safe SQL and a working pipeline with one property unproven.	Queries work but safety or idempotence is untested.	String-formatted SQL, missing WHERE on a destructive statement, or a pipeline that duplicates rows on a second run.
Total	20				

Course outcome attainment

Course outcome	Marks carried
CO5	20
Total	20

Academic integrity

Individual submission. Identical submissions score zero for both parties.